

Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий

Кафедра вычислительной техники

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №4

Преподаватель

Студент

КИ22-07Б, 032215583
номер группы, зачетной книжки

подпись, дата

подпись, дата

Д. С. Романова

инициалы, фамилия

А. М. Коробков

инициалы, фамилия

СОДЕРЖАНИЕ

Введение.....	3
Ход работы.....	4
Заключение	8
Приложение А	9

ВВЕДЕНИЕ

Описание полученного задания:

Выполняется разработка модульных тестов для программы, разработанной при выполнении работы 3.

Модульные тесты оформляются в виде отдельного проекта (подпроекта) и размещаются в репозитории.

Модульное тестирование включает разработку тестов на успешные проверки и тестирование исключительных ситуаций (например, неправильный ввод данных)

Отчёт предоставляется в электронном виде. Он должен содержать описание реализованных тестовых наборов.

Ход работы

Для выполнения практической работы №4 необходимо описать модульные тесты, выделив под них отдельный подпроект. Для этого в прошлой практической работе был создан проект `orpro_task1.tests`, в котором были описаны модульные тесты. В модульные тесты входят как позитивные тесты, так и негативные (обработка исключений). На рисунках 1 и 2 представлен пример исходного кода описания модульных тестов.

```
TEST_CLASS(AnimalTests){public : TEST_METHOD(TestAddToArray){  
    std::vector<std::unique_ptr<Animal>> testArray;  
  
    funcAddToArray<Animal>(funcAddVect("Fish Karp 1", ' '), testArray);  
    funcAddToArray<Animal>(funcAddVect("Bird Scopa 10.2", ' '), testArray);  
    funcAddToArray<Animal>(funcAddVect("Insect Scopa 10 10.10.2000", ' '), testArray);  
  
    Assert::IsTrue(typeid(*testArray[0]) == typeid(Fish));  
    Assert::IsTrue(typeid(*testArray[1]) == typeid(Bird));  
    Assert::IsTrue(typeid(*testArray[2]) == typeid(Insect));  
}  
TEST_METHOD(TestInvalidAddToArray)  
{
```

Рисунок 1 – Позитивные модульные тесты

```
TEST_METHOD(TestInvalidAddToArray)  
{  
    auto laFuncAddToArray =  
        [](const std::vector<std::string>& testTokens, std::vector<std::unique_ptr<Animal>>& testArray)  
        { return [testTokens, &testArray]() { return funcAddToArray<Fish>(testTokens, testArray); }; };  
  
    std::vector<std::unique_ptr<Animal>> testArray;  
  
    Assert::ExpectException<std::invalid_argument>(laFuncAddToArray(funcAddVect("Karp 4", ' '), testArray));  
}
```

Рисунок 2 – Негативные модульные тесты

В ходе выполнения работы было описано 16 тестов для всех публичных методов существующих классов, что обеспечивает полное покрытие функционала тестами. Также в ходе работы были использованы блоки `try-catch` для использования исключений внутри программы. На рисунке 3 представлен переписанный метод с использованием `try-catch` блоков.

```

int Parser::parseDate(const std::string& date)
{
    if (date.size() != 10 || date[2] != '.' || date[5] != '.')
    {
        throw std::invalid_argument("Incorrect format of date!");
    }

    int day, month, year;
    try
    {
        day = std::stoi(date.substr(0, 2));
        month = std::stoi(date.substr(3, 2));
        year = std::stoi(date.substr(6, 4));
    }
    catch (const std::invalid_argument&)
    {
        throw std::invalid_argument("Incorrect value of date!");
    }
    catch (const std::out_of_range&)
    {
        throw std::out_of_range("Too large value of date!");
    }

    if (!Parser::ValidateDate(day, month))
    {
        throw std::invalid_argument("Incorrect value of day|month!");
    }

    return year * 10000 + month * 100 + day;
}

```

Рисунок 3 – Использование try-catch

На рисунке 4 представлен результат выполнения тестов внутри VS.

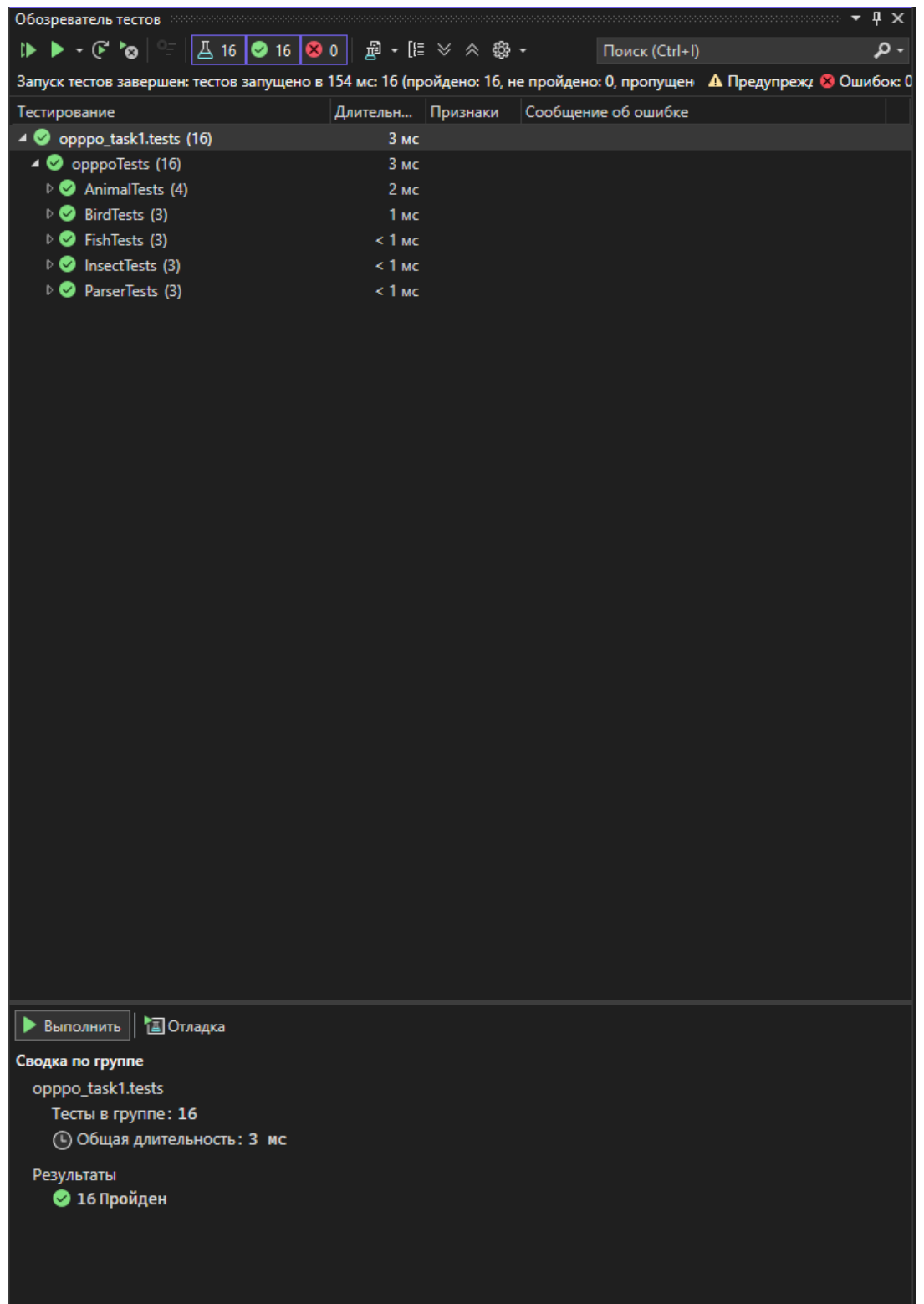


Рисунок 4 – Результат прохождения тестов

На рисунке 5 представлена сборка и выполнения тестов внутри workflow, описанном в практической работе №3.

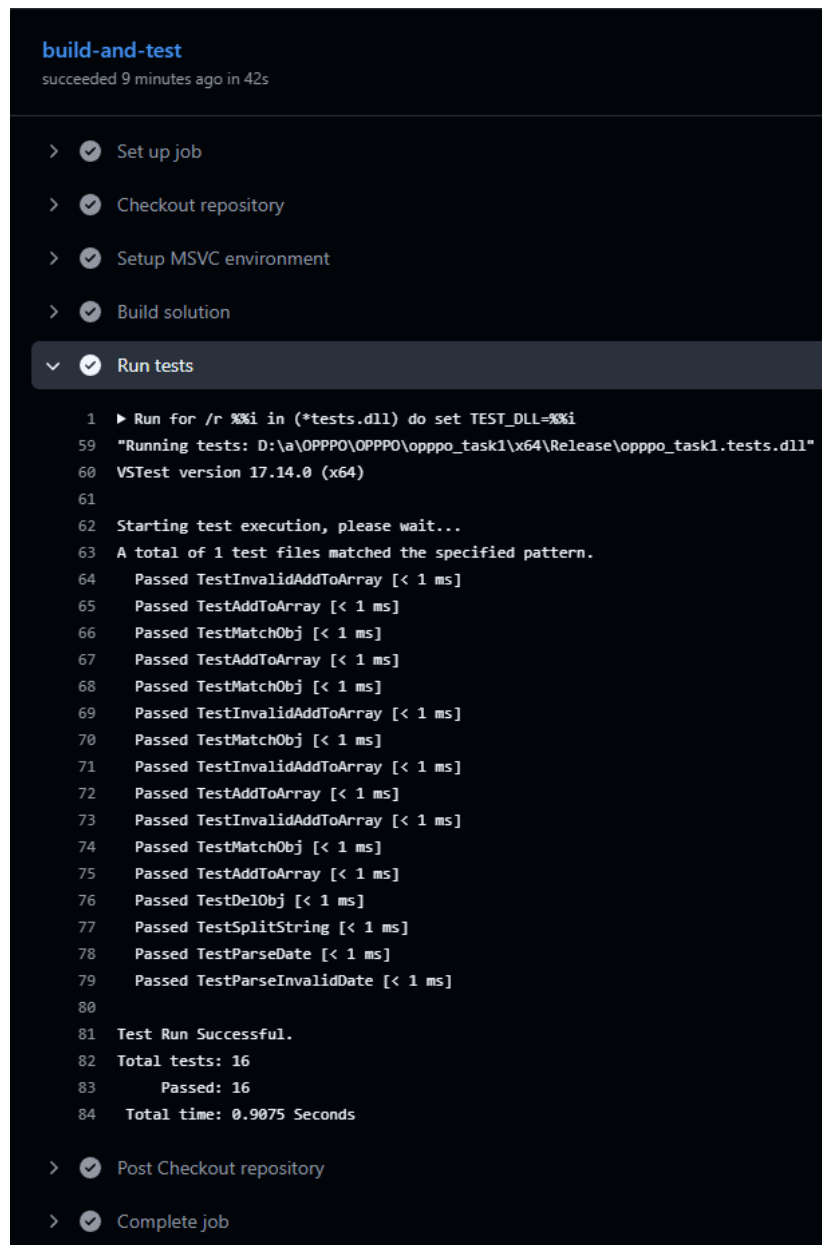


Рисунок 4 – Сборка и выполнение тестов в workflow

ЗАКЛЮЧЕНИЕ

В результате данной практической работы были закреплены навыки тестирования, написания модульных тестов.

ПРИЛОЖЕНИЕ А

1 GitHub. Система контроля версий: [сайт]. – URL:
<https://github.com/Korobkov124/OPPPPO>